

Arduino 3 — Two Analog Inputs and Serial Plotter

Tutorial Group: Arduino

Goal

Extend the program from *Arduino Analog_Input_Serial_Output* to read two analog light sensors—a photoresistor and an IR phototransistor—and visualize both signals in real time using the Arduino IDE's Serial Plotter.

In this tutorial, you will replace the potentiometer with two light-sensitive inputs. The Arduino will read the photoresistor on A0 and the IR phototransistor on A1, calculate both voltages, and send them to the Serial Plotter.

Learning objectives:

- Send numerical data to the Serial Plotter.
- Display the photoresistor and IR phototransistor voltages rather than the raw ADC values.
- Observe two analog signals changing in real time.
- Recognize the relationship between light level and the data being measured by the Arduino.

What You Need

- Arduino Mega 2560
- USB cable
- Computer with the Arduino IDE
- Breadboard and jumper wires
- Photoresistor (LDR)
- IR phototransistor
- Two 10 kΩ resistors

Part 1 — Read Two Analog Sensors

Start with the analog-reading program from *Arduino Analog_Input_Serial_Output*.

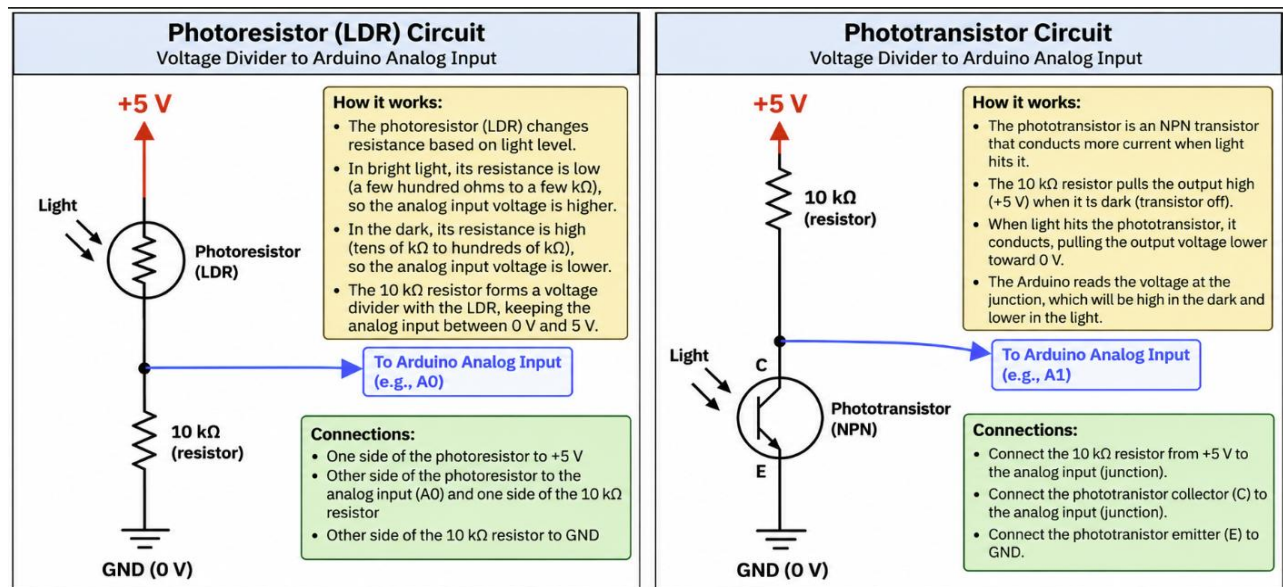
Your program should:

- Read A0 for the photoresistor.
- Read A1 for the IR phototransistor.
- Calculate and display the voltage for both inputs.

For this tutorial, plot the calculated voltages so the two sensor signals can be compared directly.

As the light level changes, the sensor voltages should change. The exact voltage and direction depend on the sensor and lighting conditions.

Build two simple light sensor circuits below:



Part 2 — Open the Serial Plotter

The Arduino IDE provides a **Serial Plotter** that can display numerical data as a graph.

Open:

Tools → Serial Plotter

The Serial Plotter uses the same Serial connection as the Serial Monitor.

Close the Serial Monitor before opening the Serial Plotter if your version of the Arduino IDE does not allow both to use the Serial connection simultaneously.

Part 3 — Plot Both Sensor Voltages

Modify your program so that it sends both calculated voltages to the Serial Plotter as shown in this example:

```
const int photoPin = A0;
const int irPin = A1;

void setup()
{
  Serial.begin(9600);
}

void loop()
{
  int photoADC = analogRead(photoPin);
  int irADC = analogRead(irPin);

  float photoVoltage = photoADC * 5.0 / 1023.0;
  float irVoltage = irADC * 5.0 / 1023.0;

  Serial.print("Photo:");
  Serial.print(photoVoltage, 2);
  Serial.print(",IR:");
  Serial.println(irVoltage, 2);

  delay(100);
}
```

Read through the program before you run it.

The declarations identify the two analog inputs, while `setup()` starts serial communication. In `loop()`, the program reads both sensors, converts the readings to volts, sends the two named voltages to the Serial Plotter, and waits 100 milliseconds. The key lines are `analogRead(photoPin)` and `analogRead(irPin)`, along with the calculations that convert the Arduino Mega 2560's analog readings to volts using the assumed 5.00v AVcc supply.

The `delay(100)` function produces about 10 samples per second, making changes easier to see.

Part 4 — Format the Serial Plotter Output

Adding Axis Marks

The Serial Plotter automatically scales the Y-axis to fit the data. To keep the graph referenced to the Arduino's 0–5.00v analog range, send fixed reference values of 0.00v and 5.00v on every line.

This can be done by adding two fixed data series to the Serial Plotter, one set to 0.00v and the other to 5.00v, to correspond with the 0.00 – 5.00v analog range. These two new traces will appear as horizontal traces.

```
Serial.print("Min:0,Max:5,");  
Serial.print("Photo:");  
Serial.print(photoVoltage, 2);  
Serial.print(",IR:");  
Serial.println(irVoltage, 2);
```

X-Axis and Sampling

The horizontal axis represents the sequence of samples over time. Each `Serial.println()` creates the next plotted sample. With `delay(100)`, the program sends about 10 samples per second.

Part 5 — Observe the Two Sensor Signals

With the Serial Plotter running, change the light reaching each sensor and observe both traces.

Observe the following:

- Confirm that the legend identifies both signals.
- The traces should remain relatively steady when the lighting conditions are stable.
- Shine both a visible light and an IR source on both sensors and compare the traces.
- Try to shield both sensors to see how sensitive they are to stray light sources in the room.
- Change the lighting quickly and observe the response.

Consider using the `map()` function you learned in *Arduino 2 — Analog Input and Serial Output* to convert one sensor's ADC reading into a useful LED blink delay.

What You Should Understand

At the completion of this tutorial, you should understand:

- How the Arduino Serial Plotter can be used to visualize changing data.
- The difference between plotting a raw ADC value and a calculated voltage.
- How an analog signal appears when viewed over time.
- That real-world measurements may contain small variations or noise.
- How software can transform a raw hardware measurement into a useful engineering quantity.
- How sensor inputs can simultaneously be displayed, analyzed, compared, and used to control an output.
- How to use a photoresistor and an IR phototransistor to measure light.

Before Moving On

- ☐ I can open and use the Arduino Serial Plotter.

- ☐ I can send numerical data to the Serial Plotter.
- ☐ I can plot calculated sensor voltages rather than raw ADC values.
- ☐ I can explain why the sensor voltages are within the Arduino's analog input range.
- ☐ I can change the light reaching the sensors and observe the corresponding changes in the graph.
- ☐ I understand that small variations in the measured voltage can occur even when the lighting is not changing.
- ☐ I can explain the difference between an ADC count and a calculated voltage.
- ☐ I can use an analog sensor input to plot the voltage and control the LED.

If you can complete these tasks and explain what you are seeing in the plot, you are ready for the next tutorial.